Computer Methods in Applied Mechanics and Engineering 418 (2024) 116546

Contents lists available at ScienceDirect

Comput. Methods Appl. Mech. Engrg.

journal homepage: www.elsevier.com/locate/cma

Physics informed WNO

Navaneeth N. [a,*], Tapas Tripura [a], Souvik Chakraborty [a,b,*]

[a] Department of Applied Mechanics, Indian Institute of Technology Delhi, Hauz Khas 110016, India
[b] Yardi School of Artificial Intelligence, Indian Institute of Technology Delhi, Hauz Khas 110016, India

ARTICLE INFO

Dataset link: https://github.com/csccm-iitd/Physics-informed-WNO

Keywords:
Operator learning
Wavelet neural operator
Physics informed learning
Stochastic projection

ABSTRACT

Deep neural operators are recognized as an effective tool for learning solution operators of complex partial differential equations (PDEs). As compared to laborious analytical and computational tools, a single neural operator can predict solutions of PDEs for varying initial or boundary conditions and different inputs without repeated independent runs from scratch. A recently proposed Wavelet Neural Operator (WNO) is one such operator that harnesses the advantage of time–frequency localization of wavelets to capture the manifolds in the spatial domain effectively. While WNO has proven to be a promising method for operator learning, the data-hungry nature of the framework is a major shortcoming. Relying completely on conventional solvers for data generation and subsequently training operators with generated data leads to a time-consuming and challenging implementation of the operators in practical applications. In this work, we propose a physics-informed WNO for learning the solution operators of families of parametric PDEs without labeled training data. The efficacy of the framework is validated and illustrated with four nonlinear spatiotemporal systems relevant to various fields of engineering and science.

1. Introduction

Natural systems are governed by certain conservation and constitutive laws. Modeling of complex constitutive laws using Partial Differential Equations (PDEs) is a prevalent approach across various corners of science and engineering disciplines [1–3]. For a complete understanding of the dynamical behavior of the underlying physical system, it is often required to solve the governing PDE model. In most cases due to the absence of analytical solutions, numerical computational tools such as Finite Element Methods (FEM) [4], isogeometric analysis [5], finite difference methods (FDM) [6], and finite volume methods (FVM) [7] are preferred. In addition to solving the system of PDEs, scientific and engineering explorations sometimes involve solving the underlying system of PDEs for different input parameters such as different domain geometry, source functions, and initial/boundary conditions (ICs/BCs), often called parametric PDEs. In this scenario, the classical numerical tools (FEM, FDM, and FVM) prove to be a computationally expensive alternative since the abovementioned schemes are highly expensive and require independent runs for each combination of input parameters. To enhance the available computational approaches data-driven surrogate approaches such as deep neural networks (DNNs) [8,9] and deep energy methods [10] are proposed. The emergence of such data-driven approaches as popular scientific machine learning (Sci-ML) techniques for learning solutions of PDEs in different areas of science and engineering is evident from the literature [11–14]. These methods have proven to be computationally superior or on par in terms of accuracy with classical schemes. The concurrent Sci-ML approaches for learning PDE solutions may be broadly categorized into two classes: data-driven and physics-informed. While the first class of approaches aims to learn the solution mapping of PDEs from the data [13,15,16], the second

* Corresponding authors at: Department of Applied Mechanics, Indian Institute of Technology Delhi, New Delhi - 110 016, India.
E-mail addresses: navaneeth.n@am.iitd.ac.in (Navaneeth N.), tapas.t@am.iitd.ac.in (T. Tripura), souvik@am.iitd.ac.in (S. Chakraborty).

https://doi.org/10.1016/j.cma.2023.116546
Received 20 August 2023; Received in revised form 13 October 2023; Accepted 13 October 2023
Available online 30 October 2023
0045-7825/© 2023 Elsevier B.V. All rights reserved.

class of approaches solves the PDE directly from the governing physics, known as Physics Informed Neural Networks (PINN) [17–19]. Although the data-driven neural networks (NNs) can learn almost everything [20,21], they require an unrealistic amount of labeled training data for effective and accurate prediction of the underlying solution. Therefore, obtaining the paired labeled training data becomes a bottleneck in data-driven NNs. Further, these approaches can learn the solution for a fixed combination of input–output only (i.e., the functional mapping between two finite-dimensional vector spaces).

To circumvent the need for labeling the training datasets, physics-informed learning was later proposed as a viable alternative to the data-driven NNs. The PINN enables the NNs to learn the solution by constraining the NN outputs to satisfy the underlying physical laws described in the form of differential equations [17,19]. While the network is trained by minimizing the total loss function, comprised of supervised boundary and initial condition loss and a residual loss computed directly from the governing equations, the derivatives involved in the residual loss are seamlessly obtained through the automatic differentiation [22]. Later, the variational-PINN was proposed, where instead of minimizing the residual of the governing differential equations, the variational energy of the system was minimized [10,23,24]. Despite effectively solving forward and inverse problems [25], PINN and variational-PINN have the same inherent limitation as data-driven NNs, i.e., the network can only be trained for a fixed set of input parameters (learning the functional mapping between two finite-dimensional vector spaces). Once the conditions or the PDE parameters are changed, a retraining of the network is inevitable. Consequently, PINN becomes an impracticable computational tool for learning the solutions of so-called parametric PDEs.

One of the plausible approaches to overcome the aforementioned shortcoming associated with the data-driven and Physics informed NNs is to employ a deep operator-based framework [26,27]. The neural operators are a class of neural networks that learn the solution of the family of parametric PDEs (i.e., learning the functional mapping between infinite-dimensional function spaces). DeepONet is one such neural operator that was introduced as a corollary of the universal approximation theorem for operators [21] in which a neural network-based framework is utilized to learn the functional mapping between the input and output paired datasets. Fundamentally, the DeepONet [28–30] is devised of two networks, which enable functional mappings between the input and output, namely branch net and trunk net. While the branch net receives the input function, the trunk net learns the corresponding output at a given sensor point. The definition of solution maps can often be redefined as learning the parameterized kernels of integral transforms from the corpus pairs of the input functions and corresponding solution space. A seminal study in a similar vein, particularly pioneering kernel-based operator learning, is highlighted in [31], where a graph neural network-based operator learning paradigm is presented. In contrast to the DeepOnet, the Graph Neural Operators (GNO) learn the kernel of integral transform through a message-passing interface between the graph networks. While GNO provides a radically different approach to operator learning, with an increase in hidden layers, the architecture becomes unstable. In more recent work, Li et al. have developed another operator learning framework called Fourier Neural Operator (FNO) [32,33], where the parameters of the integral kernel are learned in Fourier space. Here, spectral decomposition enables the framework to obtain the integral kernel in the Fourier space. The performance of FNO is promising in comparison with most of the state-of-the-art methods of operator learning. However, FNO lacks information about spatial resolution as the basis functions of FFT are frequency localized. This causes poor performance of FNO in the case of PDEs with complex geometry or, in general, in learning the spatial behavior of any signal.

Motivated by the aforementioned limitation associated with the FNO, Wavelet Neural Operator (WNO) [34–36] was proposed where the wavelet transformation [37,38] was exploited to learn the variation in the input patterns over spatial coordinates through spatial and frequency localization. Perhaps WNO can be considered a more generalized case of the previously proposed FNO. Fundamentally, WNO decomposes the input function space into detailed and approximate parts through multi-level wavelet decomposition using a wavelet basis. The outputs at lower levels of wavelet decomposition are contaminated with signal noises, whereas the outputs at higher levels preserve the relevant information. Therefore, WNO performs the parameterization of the kernel of integral transform at higher levels of multi-level wavelet decomposition, facilitating accurate and efficient learning. Though WNO is able to learn the highly nonlinear differential operators very effectively regardless of the geometry, like any other data-driven learning method, WNO also requires an impracticable amount of labeled dataset to learn the solution operator of an underlying PDE effectively. In most of the practical scenarios, the data set requires a corpus pair of input and the corresponding solution, which are often obtained from exorbitantly expensive experiments or time-consuming simulations. In this work, we propose a novel framework that integrates the benefits of WNO with the information from physics to circumvent the drawbacks of the individual frameworks. However, the incorporation of physics through governing differential equations necessitates the accurate computation of the partial derivative of the solution with respect to inputs within kernel-based NNs, which often becomes a major hurdle. We overcome this by evaluating the partial derivatives through a stochastic projection-based gradient estimation scheme, which is recently proposed in [39]. The proposed approach, referred to as the physics-informed wavelet neural operator (PIWNO), is directly trained from the governing physics; this allows training without labeled data (achieving 100% data efficiency). Unlike conventional physics-informed NNs, the proposed PIWNO learns a family of parametric PDEs, thereby accommodating an arbitrary set of input parameters, ICs, and BCs without retraining the network. Backed by the governing physics, the proposed PIWNO also generalizes to unseen input parameters. Some key features of the proposed approach include:

- The proposed PIWNO learns the solution operator of a family of PDEs, i.e., the functional mapping between infinite-dimensional function spaces. Therefore, a single PIWNO is capable of predicting the solutions of the PDEs for different initial conditions, boundary conditions, and source terms without the requirement of retraining.
- For learning solutions to PDEs, the PIWNO framework requires no labeled data; instead, a residual loss using the governing physics is formulated for functional mapping.
- Derivatives of the neural operator involved in the training loss are efficiently achieved through stochastic projection-based gradients.

We have illustrated the performance of the proposed PIWNO on three distinct examples commonly used in modeling transport and fluid dynamics, wave propagation in neurons, static electricity, and phase-field modeling. With the hands-on features, the proposed PIWNO has the potential to accelerate scientific explorations in the areas of computational physics, computational mechanics, computational biology, and other areas of engineering. Quantitatively, a trained PIWNO is about 10 to 35 times faster than traditional FEM and FDM-based PDE solvers for the problems presented in this paper.

The remainder of the paper is organized as follows. The general problem statement is described in Section 2. The proposed approach is elucidated in the following Section 3. Subsequently, Numerical examples are presented in Section 4. Finally, the conclusion and final notes are provided in Section 5.

2. Problem statement

The primary goal of this research is to provide a framework for operator learning using WNO that learns directly from governing physics. Here the physics-informed WNO seeks to learn a family of parametric PDEs rather than a single PDE. We proceed further by considering the function spaces, $\mathcal{A}$ and $\mathcal{U}$, containing all the inputs and outputs $\boldsymbol{a} \in \mathcal{A}$ and $\boldsymbol{u} \in \mathcal{U}$, respectively. Let there exist a differential operator $\mathcal{N}$ that maps the function spaces to null space, i.e., $\mathcal{N} : \mathcal{A} \times \mathcal{U} \mapsto \mathcal{O}$, where $\mathcal{O}$ is the null space. Within these function spaces a family of parametric PDEs takes the form:

$$\mathcal{N}(\boldsymbol{a}, \boldsymbol{u}) = \boldsymbol{0}, \quad \text{in } D \subset \mathbb{R}^d. \tag{1}$$

Here, the PDE is defined on an d-dimensional bounded domain, $D \in \mathbb{R}^d$, with a boundary ∂D, where the boundary condition is expressed as:

$$\boldsymbol{u} = \boldsymbol{g}, \quad \text{in } \partial D. \tag{2}$$

The parameter $\boldsymbol{a} \in \mathbb{R}^a$ denotes the input function space and $\boldsymbol{u} \in \mathbb{R}^u$ denotes the solution space of the parametric PDE. For a fixed domain $x \in D$, the input function space of the operator contains the source term $f(x,t) : D \mapsto \mathbb{R}$, the initial condition $u(x,0) : D \mapsto \mathbb{R}$, and the boundary conditions $u(\partial D, t) : \partial D \mapsto \mathbb{R}$, while the output function space comprises the solution of the parametric PDE, $u(x,t) : D \mapsto \mathbb{R}$, with t being the time coordinate. For the differential operator $\mathcal{N}$ and above input features, there exists an integral operator $\mathcal{D} : \mathcal{A} \mapsto \mathcal{U}$, which maps the input functions to the solution space. As in the case of data-driven WNO, once we have access to the N number of corpus pairs of observations of input and output $\{\boldsymbol{a}_j, \boldsymbol{u}_j\}_{j=1}^N$, a surrogate modeling approach can be employed to approximate the operator. In the context, formulation yields the nonlinear integral operator, $\mathcal{D}$, approximated by the neural network as:

$$\mathcal{D} : \mathcal{A} \times \boldsymbol{\theta}_{NN} \mapsto \mathcal{U}, \tag{3}$$

where $\boldsymbol{\theta}_{NN}$ represents the trainable parameter space of the neural network. Here, we note the underlying assumption for the operator learning that for any $\boldsymbol{a} \in \mathcal{A}$, there exists a unique solution $\boldsymbol{u} = \boldsymbol{\theta}_{NN}(\boldsymbol{a}) \in \mathcal{U}$. With N samples of input–output paired data, the appropriate loss function for a pure data-driven framework can be formulated as follows:

$$\mathcal{L}_{\text{data}} \left(\boldsymbol{u}, \boldsymbol{\theta}_{NN}(\boldsymbol{a})\right) = \left\|\boldsymbol{u} - \boldsymbol{\theta}_{NN}(\boldsymbol{a})\right\|_{\mathcal{U}}^2 = \int_D \left|u(x_i) - \boldsymbol{\theta}_{NN}(a)(x_i)\right|^2 \, \mathrm{d}x. \tag{4}$$

The operator loss, evaluated by the averaging error across all possible inputs, and n_d space discretizations, is expressed as:

$$\mathcal{L}_{\text{data}} \left(\boldsymbol{u}, \boldsymbol{\theta}_{NN}(\boldsymbol{a})\right) = \frac{1}{N} \sum_{j=1}^{N} \sum_{i=1}^{n_d} \left|u_j(x_i) - \mathcal{D}\left(a_j, \boldsymbol{\theta}_{NN}\right)(x_i)\right|^2. \tag{5}$$

Thus optimizing the network for the given loss function yields the optimal network parameters:

$$\boldsymbol{\theta}_{NN}^* = \underset{\boldsymbol{\theta}_{NN}}{\operatorname{argmin}} \, \mathcal{L}_{\text{data}} \left(\boldsymbol{u}, \boldsymbol{\theta}_{NN}(\boldsymbol{a})\right). \tag{6}$$

Now, to implement the physics-informed learning, we consider a differential operator $\mathcal{N}$ corresponding to the Eq. (1), expressed in the residual form as:

$$\mathcal{N}(\boldsymbol{a}, \boldsymbol{u}; \boldsymbol{\theta}_{NN}) = 0, \tag{7}$$

where $\mathcal{N}$ contains all the derivatives of space and time, functions of $\boldsymbol{x}, t$ and $\boldsymbol{u}$, and input parameters γ. Thus Eq. (7) can be rewritten as:

$$\mathcal{N}\left(\mathbf{x}, t, \boldsymbol{u}, \partial_{\mathbf{t}} \boldsymbol{u}, \partial_{\mathbf{t}}^2 \boldsymbol{u} \ldots, \partial_x \boldsymbol{u}, \partial_t^n \boldsymbol{u}, \ldots, \partial_x^n \boldsymbol{u}, \boldsymbol{\gamma}\right) = 0. \tag{8}$$

Here, the constraint in Eq. (7) is evaluated using the PDE loss along with boundary loss and initial condition loss. Employing a mean squared loss function, the expression of PDE loss $\mathcal{L}_{\text{pde}}$ follows the form

$$\mathcal{L}_{\text{pde}} \left(\mathcal{D}(\boldsymbol{a}, \boldsymbol{\theta}_{NN})\right) = \underbrace{\left\|\mathcal{N}(\boldsymbol{a}, \boldsymbol{u}; \boldsymbol{\theta}_{NN})\right\|_D^2}_{\text{Physics Loss}} + \alpha \underbrace{\left\|\boldsymbol{u}(\boldsymbol{x}; \boldsymbol{\theta}_{NN}) \mid_{\partial D} - \boldsymbol{g}\right\|^2}_{\text{Boundary Loss}}, \tag{9}$$

where $\alpha \in \mathbb{R}$ represents the participation factor of the approximation error at boundary points in the total PDE loss. For N training input samples, with n_d space discretizations and n_b boundary points, Eq. (9) can be rewritten as

$$\mathcal{L}_{\text{pde}}\ \left(\mathcal{D}(\boldsymbol{a}, \boldsymbol{\theta}_{NN})\right) = \frac{1}{N}\sum_{j=1}^{N}\sum_{i=1}^{n_d}\left|\mathcal{N}(a_j, u_j; \boldsymbol{\theta}_{NN})(x_i)\right|^2 + \alpha\sum_{j=1}^{N}\sum_{i=1}^{n_b}\left|u_j(x_i; \boldsymbol{\theta}_{NN}) - g(x_i)\right|^2. \tag{10}$$

As was mentioned, the residual loss utilizes the stochastic projection-based gradients to obtain the derivatives of the output $\boldsymbol{u}$. The derivative of $\boldsymbol{u}$ at a given point $\bar{\boldsymbol{x}}$ is computed by

$$\frac{\partial \boldsymbol{u}(\bar{\boldsymbol{x}}, \boldsymbol{\theta}_{NN})}{\partial \boldsymbol{x}} = \frac{\frac{1}{N_b}\sum_{i=1}^{N_b}(\boldsymbol{u}(\boldsymbol{x}_i, \boldsymbol{\theta}_{NN}) - \boldsymbol{u}(\bar{\boldsymbol{x}}, \boldsymbol{\theta}_{NN}))(\boldsymbol{x}_i - \bar{\boldsymbol{x}})^T}{\frac{1}{N_b}\sum_{i=1}^{N_b}(\boldsymbol{x}_i - \bar{\boldsymbol{x}})(\boldsymbol{x}_i - \bar{\boldsymbol{x}})^T}, \tag{11}$$

where $\boldsymbol{x}_i = \{x_i, y_i\}$ is considered to be a generic neighborhood point and N_b represents the number of neighborhood points.

3. Proposed methodology

While the wavelet neural operator learns the solution of the family of parametric PDE, physics-informed learning aims to learn the operator from physics constraints. Moreover, since the operator is trained with loss functions computed directly from the governing equations $\mathcal{L}_{\text{pde}}$, one needs to evaluate the derivatives operator output involved in the governing physics. However, in practice, it is non-trivial to obtain the derivatives of the output field, especially when the architecture of θ_{NN} is devised of the convolution layers. Thus, we conveniently employ stochastic projections to compute the gradients involved in the training loss. In this section, we provide a high-level description of the WNO and stochastic projection-based gradients.

3.1. Wavelet neural operator

The WNO is a data-driven neural operator for learning the integral operator of a family of parametric PDEs. In parametric PDEs, we denote the input and output function spaces by $\mathcal{A}$ and $\mathcal{U}$. The input function space $\mathcal{A}$, in general, contains different PDE parameters, geometries, and initial and boundary conditions, while the output space $\mathcal{U}$ denotes the solution space. Mathematically, we consider a smooth d-dimensional domain D over which the functional spaces are constructed. We define the input–output pair $\{a(x) \in \mathcal{A},\ u(x) \in \mathcal{U}\}$ such that $u(x) = \mathcal{D}(a(x))$. Over the domain, we take a relatively simple example of the form,

$$\begin{aligned} -\partial_{xx}u(x) + \omega u(x) &= f\left(x, u(x)\right); \quad x \in D, \\ u(x) &= 0; \quad x \in \partial D, \end{aligned} \tag{12}$$

where ω is the parameter of the PDE and $f : D \times \mathbb{C} \to \mathbb{C}$ is a continuous function. For a valid value of ω the above differential equation may be recast as an integral equation of the following form,

$$u(x) = \int_D k(x, \xi) f\left(\xi, u(\xi)\right) d\xi; \quad x \in D, \tag{13}$$

where $k(\cdot) \in \mathbb{C}$ is continuous and non-negative Green's function. For nonlinear parametric PDEs, the above integral equation may be generalized to Urysohn-type integral equations,

$$u(x) = \int_D k(x, \xi) f\left(\xi, u(\xi)\right) d\xi + g(x); \quad x \in D, \tag{14}$$

where $k(\cdot)$ is the kernel of the nonlinear integral equation representing the nonlinear counterpart of Green's function, and g is some linear transformation. The above integral equation is valid only for input source function $f(x, u(x))$. However, neural operators generalize the above nonlinear integral equation for any general input function $a(x)$. Therefore, our aim is to learn the nonlinear integral operator $\mathcal{D} : a(x) \mapsto u(x)$, which maps the input function space $a(x) \in d_a$ to the solution space $u(x) \in d_u$. From Eq. (14) the input space $a(x)$ is a tuple of the spatial coordinate x, the initial conditions $u(x)$, and the forcing function $f(x, u(x))$. The data-driven WNO requires a collection of N pairs (number of training samples) of the input–output $\{a_j, u_j\}_{j=1}^{N}$. In neural network setup, the approximated integral operator $\mathcal{D}$ is given as,

$$\mathcal{D} : \mathcal{A} \times \boldsymbol{\theta}_{NN} \mapsto \mathcal{U}. \tag{15}$$

Since the integral in Eq. (14) cannot be defined in an infinite dimensional space, therefore, a finite-dimensional parameterization space is obtained by discretization of the solution domain $D \in \mathbb{R}^n$. For featuring a multi-dimensional kernel convolution, the input $a(x)$ is lifted to some high dimensional space d_v using a local transformation $\mathrm{P} : a(x) \mapsto v_0(x)$. The local transformation P can be modeled as a shallow fully connected neural network (FNN) or as a 1×1 convolution. On the lifted space, l-number of iterations of the form in Eq. (14) is performed. The iterations are featured by the transformation $G : \mathbb{R}^{d_v} \mapsto \mathbb{R}^{d_v}$ such that $v_{j+1} = G(v_j)$. These iterations resemble the usual concepts of hidden layers in neural networks; however, the hidden layers are defined to follow the operator theory in functional analysis [40]. At the end of the l-iterations, a second local transformation $\mathrm{Q} : v_l(x) \mapsto u(x)$ is applied to obtain the final solution space $u(x) \in \mathbb{R}^{d_u}$. Motivated by the Eq. (14), the step-wise updates $G(\cdot)$ is defined as follows,

$$G(v_j)(x) := \varphi\left(\left(K(a; \phi) * v_j\right)(x) + W v_j(x)\right); \quad x \in D, \quad j \in [1, l], \tag{16}$$

where $\varphi(\cdot)\mathbb{R} \mapsto \mathbb{R}$ is a non-linear activation function, $\phi \in \theta_{NN}$ is the kernel parameters, $W : \mathbb{R}^{d_v} \mapsto \mathbb{R}^{d_v}$ is a linear transformation, and K is the nonlinear integral operator. Here the integral operator K is defined as,

$$\left(K(a;\phi) * v_j\right)(x) := \int_D k\left(a(x), x, \xi; \phi\right) v_j(\xi) \mathrm{d}\xi; \quad x \in D, \quad j \in [1, l]. \tag{17}$$

where $k\left(a(x), x, \xi; \phi\right)$ denotes the kernel of the nonlinear integral equation in Eq. (14). The aim is to learn the kernel $k\left(a(x), x, \xi; \phi\right)$ by parameterizing the neural network in the wavelet space. To create the parameterization space in the wavelet domain, wavelet transform is performed on the lifted input $v_j(x)$. The forward and inverse wavelet transforms $\mathcal{W}(\cdot)$ and $\mathcal{W}^{-1}(\cdot)$ are defined as follows [41],

$$\begin{aligned}(\mathcal{W}v_j)(s,\tau) &= \int_D \Gamma(x) \frac{1}{|s|^{1/2}} \psi\left(\frac{x-\tau}{s}\right) dx, \\ (\mathcal{W}^{-1}(v_j)_w)(x) &= \frac{1}{C_\psi} \int_0^\infty \int_D (v_j)_w(s,\tau) \frac{1}{|s|^{1/2}} \tilde{\psi}\left(\frac{x-\tau}{s}\right) d\tau \frac{ds}{s^2},\end{aligned} \tag{18}$$

where $\psi(x)$ denotes the orthonormal mother wavelet, and s and τ are the scaling and translational parameters of wavelet decomposition, $(v_j)_w$ is the wavelet decomposed coefficients of $v_j(x)$, $\psi(\cdot)$ is the scaled and shifted mother wavelet, and $0 < C_\psi < \infty$ is the admissible constant [41]. Since the aim is to learn the kernel integration in the wavelet domain, the kernel k_ϕ is directly defined in the wavelet space, denoted as $R_\phi = \mathcal{W}(k_\phi)$. Using the convolution theorem, the integral in Eq. (17) over the wavelet domain is expressed as

$$\left(K(\phi) * v_j\right)(x) = \mathcal{W}^{-1}\left(R_\phi \cdot \mathcal{W}(v_j)\right)(x); \quad x \in D. \tag{19}$$

During training the network, performing the complex wavelet decomposition in Eq. (18) is expensive since the scale and translation parameters s and τ can be infinite-dimensional. In order to perform the complex wavelet transform quickly, we utilized the slim dual-tree complex wavelet transform (DTCWT) toolbox in [42], originally proposed in [43]. At each decomposition level, the slim DTCWT provides two sets (real and imaginary) of six wavelet coefficients, which roughly represent coefficients of 15°, 45°, 75°, 105°, 135°, and 165° wavelets. Due to conjugate symmetry, the coefficients are halved at each decomposition level. To learn the relevant features of the input, the parameterization of the kernel $R_\phi(\cdot)$ is done at the highest level of the DTCWT. Thus, if d_l denotes the dimension of each wavelet coefficient at the last level of DTCWT, then for the input $v_j(x)$, the decomposed output $\mathcal{W}\left(v_j;\ell\right)$ will have a dimension $d_l \times d_v$. Thereafter the weight tensor $R_\phi(\ell)$ is constructed with the size $d_l \times d_v \times d_v$. The kernel convolution $(R_\phi \cdot \mathcal{W}(v_j))(x)$ follows,

$$\left(R \cdot \mathcal{W}(v_j;\ell)\right)_{t_1,t_2} = \sum_{t_3=1}^{d_v} R_{t_1,t_2,t_3} \mathcal{W}(v_j;\ell)_{t_1,t_3}; \quad I_1 \in [1, d_l], \quad I_2, I_3 \in d_v. \tag{20}$$

where the integration is performed in the uplifting dimension. In total, the DTCWT provides 12 wavelet coefficients, real and imaginary coefficients of 15°, 45°, 75°, 105°, 135°, and 165° wavelets. There, to learn the parametric space, we need twelve weight tensors and twelve convolutions defined in Eq. (20).

The WNO is defined by the following parameters (i) uplifting dimensions, (ii) choice of vanishing moments in the wavelets, (iii) decomposition level, and (iv) the number of wavelet blocks. The uplifting dimension denotes the number of kernels. As in convolution neural networks (CNN), a higher uplifting dimension makes it easy for the network to learn, but too high may lead to overfitting. The vanishing moments represent the smoothness of a wavelet. Ideally, for an input having high spatial variations, wavelets with low-order (in WNO, < 4) vanishing moments help in capturing the local features. For a relatively smooth image, a wavelet with higher (> 4) vanishing should be utilized. The number of wavelet decomposition levels ℓ depends on the kernel size. If fine features are to be captured, then a smaller kernel is required, meaning a higher decomposition level. It is to be noted that each decomposition subsamples the input size by a factor of two. For e.g., if the input has a spatial dimension 64×64 and if we want a kernel of sizes 4 and 8, then we need a decomposition level of 4 and 3, respectively. The number of wavelet blocks depends on the complexity of the underlying operator to be learned. A schematic representaion of the overall architecture of the proposed PIWNO is shown in the Fig. 1.

3.2. Derivatives through stochastic projection

As we have discussed earlier, the primary requirement to compute L_{pde}, which is utilized to train the network, is the evaluation of derivatives of output $\boldsymbol{u}$. However, it is not straightforward to compute the derivatives for neural operators efficiently. While there are different methods available to compute the gradients, here we employ stochastic projection-based gradients to obtain the gradients seamlessly. The efficacy of the method is elucidated in [39]. In principle, the stochastic projection method can be understood as the upscaling of microscopic information in a multi-scale formulation where the micro-scale information accounts for non-trivial directional information of the evolving state variables. Now, to put the stochastic projection in perceptive of physics-informed operator learning, we consider a domain with a finite number of collocation points over which the solution is sought. Also, there is a neighborhood defined by the distance for each collocation point. Now we assume that the variation of the field variable is distinctively measured only up to a certain characteristic distance while the variation of the field variable less than the characteristic length can be treated to be a stochastic process. While in the former case, the field variable is considered to be a macroscopic field

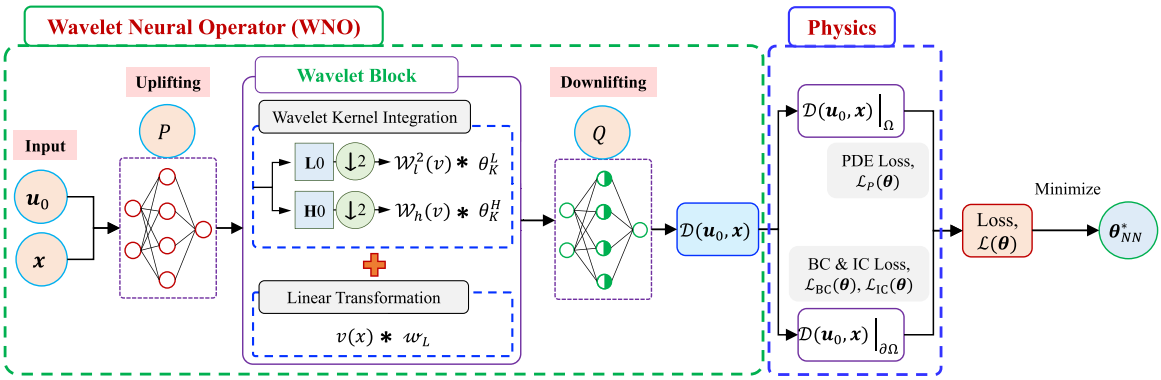

Fig. 1. Physics-informed Wavelet Neural Operator (PIWNO). Motivated by the operator theory in classical functional analysis the WNO architecture aims to learn Green's function of underlying PDEs. In WNO the inputs are first lifted to a high-dimensional latent space, over which certain iterations followed by nonlinear activation are applied. The iterations are represented using wavelet kernel integration blocks. It consists of a kernel integration network that learns the kernel of integration and a linear transformation network that learns the kernel integration constants. For learning the kernel, the latent inputs are localized in space-frequency using wavelets, which is a feature of WNO. The integration outputs and the integral constants are added and finally down-lifted to obtain the solution of underlying PDE. The solutions are constrained to satisfy given PDEs, BC, and IC. For devising the PDE constraint, the spatial derivatives are computed through a stochastic projections-based gradient estimation scheme.

variable, the latter one is considered to be the microscopic field variable. Similarly, the field variables can be differentiated into those which evolve slowly with time and those which evolve on faster time scales based on the characteristic time interval.

To begin with the formulation, we suppose $\boldsymbol{u}(\boldsymbol{x})$ to be a macroscopic field variable prior to conditioning based on any microscopically inspired information. Further, to characterize the macroscopic field variable $\boldsymbol{u}(\boldsymbol{x})$ by means of neighborhood information, a zero mean noise term is added to the $\boldsymbol{u}(\boldsymbol{x})$, which results in field measurement, $\boldsymbol{u}(\boldsymbol{z})$ at $\boldsymbol{z}$ such that $\boldsymbol{z} \neq \boldsymbol{x}$, and is expressed as:

$$\boldsymbol{u}(\boldsymbol{z}) = \boldsymbol{u}(\boldsymbol{x}) + \boldsymbol{\Delta}\eta. \tag{21}$$

Here, the term $\boldsymbol{\Delta}\eta$ represents noise due to the unaccounted fluctuations in the microscopic level. The noisy observation at the macroscopic level, after constraining the information from the micro scale, sampled at a time t, can be written in the following form;

$$d\boldsymbol{Z}_t = \boldsymbol{h}(\boldsymbol{x}_t, \boldsymbol{z}_t)dt + \sigma d\boldsymbol{W}_t. \tag{22}$$

Here we note that, in the above expression, $\boldsymbol{h}(\cdot,\cdot)$ is a function that yields the difference in the field variable values at points separated by a distance observed in macroscopic level. While the term $\sigma d\boldsymbol{W}_t$ indicates noise that is reliant on its microscopic counterpart and microscopically sampled function $\boldsymbol{Z}_t$, $\boldsymbol{W}_t$ is Brownian motion that is independent of $\boldsymbol{\eta}_t$. To obtain the microscopically informed spatial variation in the field variable, a conditional expectation of $\boldsymbol{u}$ is utilized. Consequently, applying the analogy of stochastic filtering on microscopically sampled function $\boldsymbol{Z}_t$ for a process measurable in a given probability space and further simplification utilizing the Kallianpur–Striebel formula and Radon–Nikodym derivative used in the change of measure, the $(\boldsymbol{z}_t - \boldsymbol{x}_t)$ yields the following form:

$$(\boldsymbol{z}_t - \boldsymbol{x}_t) = (\boldsymbol{z}_{t_0} - \boldsymbol{x}_{t_0}) + \int_{\hat{t}_0}^{\hat{t}} \left(\pi_s\left((\boldsymbol{z}-\boldsymbol{x})\boldsymbol{h}^T\right) - \pi_s(\boldsymbol{h})^T \pi_t(\boldsymbol{z}-\boldsymbol{x})\right) \cdot \left(\sigma\sigma^T\right)^{-1} \left(d\boldsymbol{z}_t\right), \tag{23}$$

where $\boldsymbol{z}_{t_0} - \boldsymbol{x}_{t_0} = \Delta$ with t_0 being the initial time. The term $\pi_t(\boldsymbol{z}-\boldsymbol{x})$ represents conditional expectation with respect to given probability space $\mathcal{P}$ and which is expressed by $\Pi_t(\boldsymbol{u}) = E_{\mathcal{P}}[\boldsymbol{u}(\boldsymbol{x})|\mathcal{F}_t]$ such that $\mathcal{F}_t$ denotes the sequence formed by adding up Z_t at each time till t. In addition, Eq. (23) also satisfies the presumption that the macroscopic field variable cannot be resolved into a vector having length less than the characteristic length, $|\Delta|$. Here, the function, $\boldsymbol{h}(\boldsymbol{x},\boldsymbol{z})$ is smooth enough and satisfies

$$\boldsymbol{h}(x,z) = \begin{cases} 0, & \text{if } |z-x| \leq 0 \\ Nonzero, & \text{otherwise.} \end{cases} \tag{24}$$

It is important to notice that $\boldsymbol{Z}_t$ in Eq. (23) denotes the specific observation: $\boldsymbol{Z}_t = \boldsymbol{Z}_{t'} + \int_{\delta\hat{t}} \Delta d\hat{s}$ where as t' is the previously sampled macroscopic time.

From Eq. (22), variance of the microscopic observation can be substituted as: $\boldsymbol{\sigma\sigma}^T \equiv \pi_t\left(\boldsymbol{h}\boldsymbol{h}^T\right)\delta\hat{t}$. Since the integration is carried out within the least microscopically resolvable time period t, macroscopic level temporal variations in the integrand are not solvable. Thus, $\boldsymbol{z}_t - \boldsymbol{x}_t$ is considered to be drift-less and can be approximated as:

$$(\boldsymbol{z}_t - \boldsymbol{x}_t) \sim \Delta + \mathbf{G}\Delta, \tag{25}$$

Table 1
Results of the prediction error of the physics informed WNO for all the illustrated examples. The prediction error corresponds to the relative Mean Square Error (MSE) of the optimized model after training, computed over the test data. N_s represents the number of training samples.

	PDEs	WNO		
		PIWNO	Data-driven	Data + physics
Relative MSE	Burger's	$0.0518 \pm 0.041\%$	$0.1552 \pm 0.17\%$	$0.0362 \pm 0.028\%$
N_s		–	300	300
Relative MSE	Nagumo	$0.0778 \pm 0.088\%$	$0.0254 \pm 0.023\%$	$0.0202 \pm 0.020\%$
N_s		–	800	800
Relative MSE	Non-homogeneous	$0.0469 \pm 0.015\%$	$0.0549 \pm 0.020\%$	0.0371 ± 0.020
N_s	Poisson's	–	500	500
Relative MSE	Allen-Cahn	$2.73 \pm 0.98\%$	$2.98 \pm 0.83\%$	2.37 ± 0.70
N_s		–	600	600

where the expression of $\mathbf{G}$ is given by

$$\mathbf{G} = \left(\pi_t\left((\boldsymbol{z}-\boldsymbol{x})\boldsymbol{h}^T\right) - \pi_t(\boldsymbol{z}-\boldsymbol{x})\right)\pi_t(\boldsymbol{h})^T(\mathrm{Var}(\boldsymbol{h}))^{-1}. \tag{26}$$

Now to compute the gradients at a given point $\bar{\boldsymbol{x}} = \{x_p, y_p\}$ in the domain, a neighborhood is specified in terms of the radius r_n. Once the neighborhood is defined, one may choose N_t number of collocation points inside the neighborhood. Subsequently, gradient of output $\mathcal{U}$ with the input variable at $\bar{\boldsymbol{x}}$ is computed by Eq. (11), where $\boldsymbol{x}_i = \{x_i, y_i\}$ is considered to be a generic neighborhood point.

$$\hat{\mathbf{G}}(\boldsymbol{x}=\bar{\boldsymbol{x}}) = \frac{\partial \boldsymbol{u}(\bar{\boldsymbol{x}}, \boldsymbol{w})}{\partial \boldsymbol{x}} = \frac{\frac{1}{N_t}\sum_{i=1}^{N_b}(\boldsymbol{u}(\boldsymbol{x}_i, \boldsymbol{w}) - \boldsymbol{u}(\bar{\boldsymbol{x}}, \boldsymbol{w}))(\boldsymbol{x}_i - \bar{\boldsymbol{x}})^T}{\frac{1}{N_t}\sum_{i=1}^{N_b}(\boldsymbol{x}_i - \bar{\boldsymbol{x}})(\boldsymbol{x}_i - \bar{\boldsymbol{x}})^T}. \tag{27}$$

The step-by-step implementation of physics-informed WNO is given in Algorithm 1.

Algorithm 1 Stochastic Projection based physics Informed Wavelet Neural Operator.

Requirements: Boundary conditions, initial conditions, and PDE describing the physics constraint.

Initialize: Network parameters, $\boldsymbol{w} = \{w_i^s, b_i^s\}$ of the WNO, $\boldsymbol{\theta}_{NN}$.
Collect the output of WNO ($\mathcal{U}$) and corresponding coordinates of the grid points over the domain $\{x_f^i, y_f^i\} \in \Omega$.
Collect the output of WNO ($\mathcal{U}$) and corresponding coordinates of the grid points at the boundary of the domain $\{x_b^i, y_b^i\} \in \partial\Omega$.
For the given resolution of the field, define the neighborhood of each grid point points in terms of distance Δx and Δy.
Obtain the first-order gradients at all the collocation points using Eq. (27) and store the gradients.
From the first-order gradients, using the same formulation Eq. (27) the second-order gradients can be achieved.
Define the PDE loss $\mathcal{L}_{PDE}$ in terms of components of the gradients.
Define the Boundary loss $\mathcal{L}_{BC}$, Sum the all losses to get the total loss $\mathcal{L}_{total}$.
while $\mathcal{L} > \epsilon$ **do**
    Train the network: $\{w_i^s, b_i^s\} \leftarrow \{w_i^s, b_i^s\} - \delta\nabla_{\boldsymbol{w},\boldsymbol{b}}L(\boldsymbol{w}, \boldsymbol{b})$.
    epoch= epoch + 1.
Return the optimum parameters for the PINN.
Obtain Predictions/solutions.

Output: Prediction of the field variable/solution of PDE.

4. Numerical examples

In this section, we evaluate the efficacy and robustness of the framework with the benchmark numerical examples relevant to the various engineering systems. A relative Mean Square Error (MSE) of the test case instances is used to evaluate the performance of the framework. In addition, we closely examine and carry out a comparative study of three distinct training scenarios of WNO; firstly, learning solely based on physics constraints; secondly, data-driven setting; and lastly, a hybrid approach in which the WNO utilizes both data and physics. In regards to the architecture of the WNO framework, it is composed of a number of layers that varies from 3–5 in accordance with the example. The ADAM optimizer with an initial learning rate of 0.001 and a weight decay of 10^{-6} is used to optimize the WNO's parameters. A learning rate decays chosen to be 0.75 for every 50 epochs during training. While the total number of epochs for the example varies from 300 to 400, the batch size varies from 10 to 25. The results of considered numerical examples are summarized in Table 1. Additional results on time comparisons of trained PIWNO predictions with conventional solvers are provided in Appendix A.1. Moreover, the performance of trained PIWNO is evaluated on out-of-distribution predictions, and the results are summarized in Appendix A.2. Interested readers may refer to the appendix for more details.

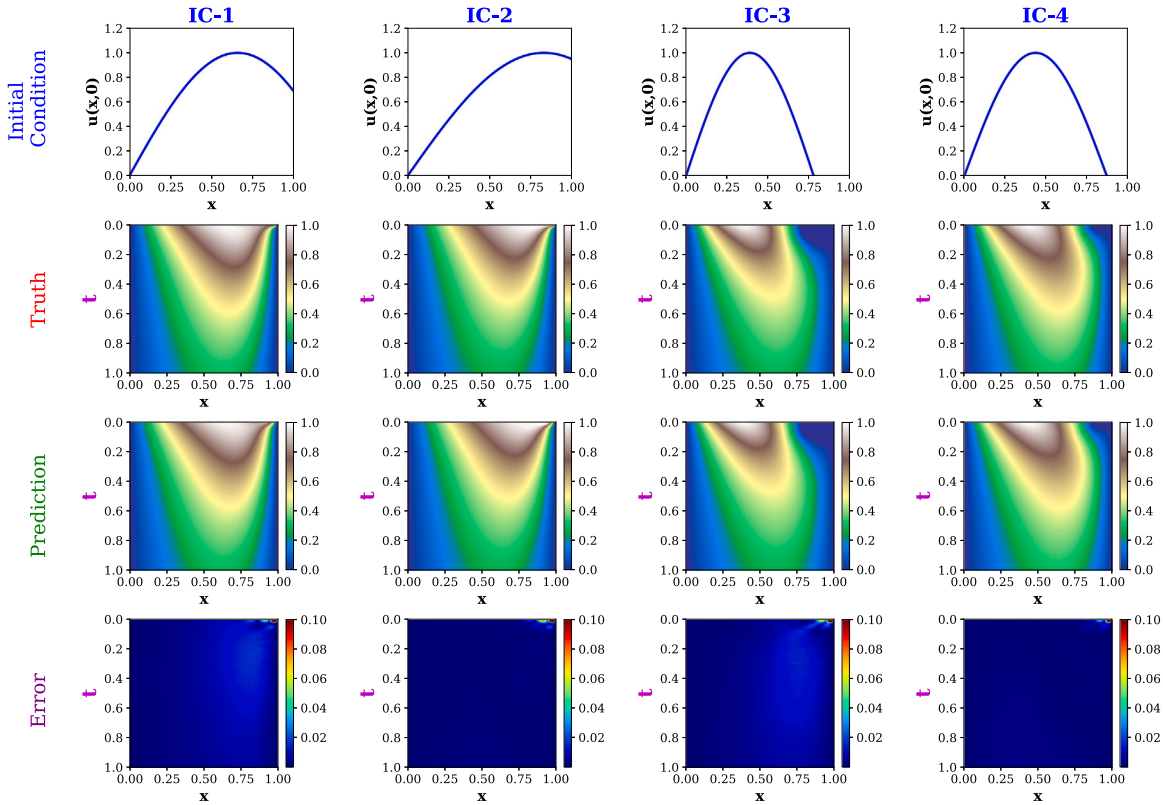

Fig. 2. The results for the parametric Burger's equation comprised of given initial conditions, corresponding ground truth solutions, predictions, and error plots illustrated with 4 different representative sample instances out of 100 test samples. The PIWNO maps the initial conditions to the corresponding solutions $u(x,t)$ over the domain with a spatiotemporal resolution of 81×81.

4.1. Burgers' diffusion dynamics

As a first example, we consider the popular Burgers' equation with varying initial conditions. The 1D Burgers equation is commonly used to mathematically model the physics of weave formation, turbulence, fluid flows in fluid mechanics and gas dynamics, traffic flow and etc. [44,45]. The one-dimensional (1D) Burgers equation with periodic boundary conditions is given by the following mathematical form:

$$\begin{aligned} \partial_t u(x,t) + \frac{1}{2}\partial_x u^2(x,t) &= \nu \partial_{xx} u(x,t), && x \in [0,1], t \in [0,1] \\ u(x=0,t) = u(x=1,t) &= 0, && t \in [0,1] \\ u(x,0) &= u_0(x), && x \in [0,1] \end{aligned} \tag{28}$$

where $\nu = 0.1$ is the viscosity of the flow, $u_0(x) = cos(\zeta \pi x) + sin(\eta \pi x)$ is the initial condition. For generating multiple initial conditions, the parameter ζ and η are simulated from uniform distributions with $\zeta \sim \text{Unif}(0.5, 1.5)$ and $\eta \sim \text{Unif}(0.5, 1.5)$. For different initial conditions, the goal here is to learn the operator, $\mathcal{D} : u_0(x) \mapsto u(x,t)$, where $u(x,t)$ is the Spatio-temporal solution at some time t. The ground truth data sets are generated employing a Matlab PDE solver, where the viscosity parameter is taken as $\upsilon = 0.1$. The resolution of the solution space is chosen to be 81×81. Upon training with randomly generated instances of the initial conditions, the model accuracy is evaluated with unseen realization, while a validation set of realizations is used to obtain the optimum tunable hyperparameters of the model.

The visualization of the predicted results for 4 representative test samples is showcased in Fig. 2. It can be apparently observed that the predictions of PIWNO yield a remarkable concordance with the corresponding reference solutions. Moreover, the results of prediction accuracy are presented in Table 1, where we compare the models of WNO in the three distinct training scenarios. While in the first case, in which the WNO is trained with physics constraints, the model yields an average relative MSE of $\approx 0.0518\%$, the data-driven WNO yields a mean relative MSE of $\approx 0.1552\%$. On the other hand, in the third case, when the models trained with both data loss and physics loss yield an average prediction error of $\approx 0.0362\%$, observed to be more accurate in comparison with WNO, which is either only physics-informed or only data-driven.

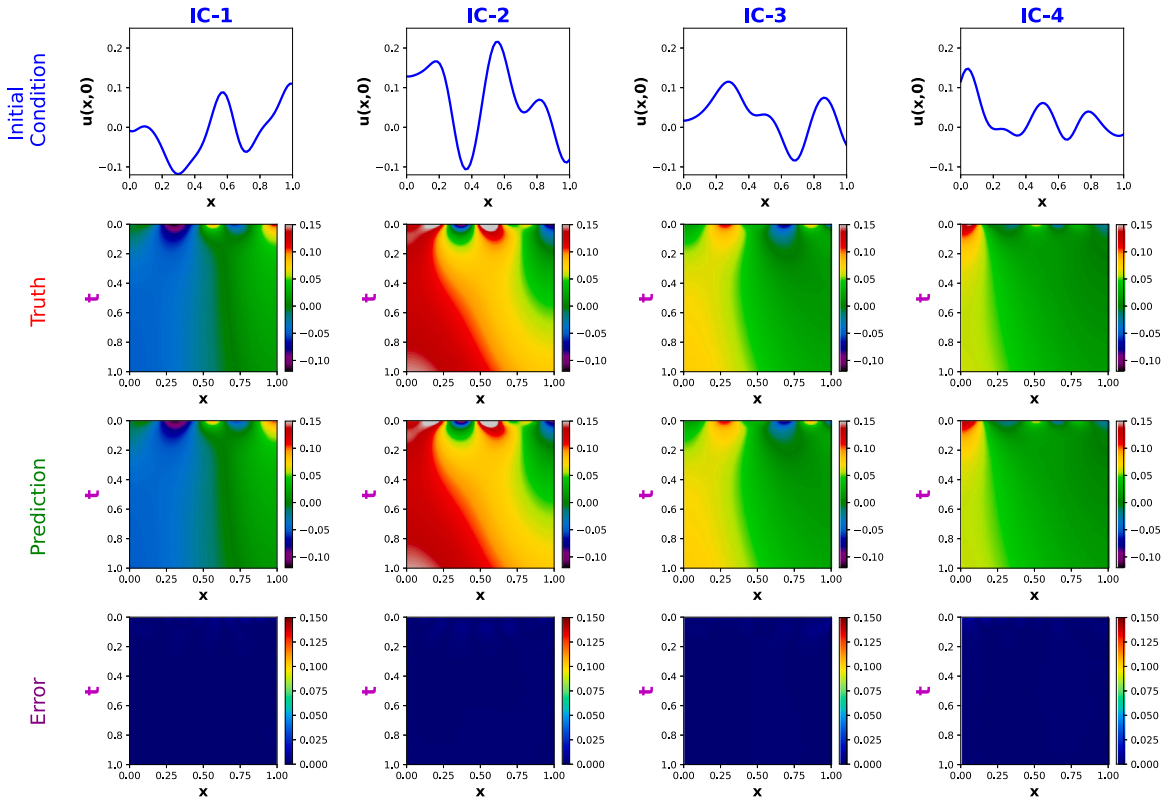

Fig. 3. The results for the parametric Nagumo equation comprised of given initial conditions, corresponding ground truth solutions, predictions, and error plots illustrated with 4 different representative sample instances out of 100 test samples. The physics-informed WNO maps the initial condition to the corresponding solution $u(x,t)$ over the domain with a spatio-temporal resolution of 65×65.

4.2. Nagumo equation

The second numerical example we consider here is the Nagumo equation. The application of the Nagumo equation can be found in modeling wave propagation in a neuron, in particular, studies involving governing dynamics of voltage across the nerve cell and the impulses in the nerve fiber. The Nagumo equation [46,47] with periodic homogeneous Neumann boundary conditions can be described as:

$$\begin{aligned} \partial_t u - \varepsilon \partial_{xx} u &= u(1-u)(u-\alpha), && x \in [0,1], t \in [0,1] \\ u(x=0,t) &= u(x=1,t) = 0, && t \in [0,1] \\ u(x,0) &= u_0(x), && x \in [0,1] \end{aligned} \tag{29}$$

where the model parameter $\alpha \in \mathbb{R}$ determines the speed of a wave traveling down the length of the axon and $\varepsilon > 0$ controls the rate of diffusion. For our study, the variables are chosen to be $\varepsilon = 1, \alpha = -1/2$. The initial conditions (vector of $\mathbb{R}^{65}$) are generated using a Gaussian random field with a kernel of the form:

$$\mathcal{K}(x,y) = \sigma^2 exp\left(\frac{-(\boldsymbol{x}-\boldsymbol{x}')^2}{2l^2}\right), \tag{30}$$

where the value of σ and the l parameters are chosen such that $\sigma = 0.1$; and $l = 0.1$. Moreover, to obtain the ground truth data, the PDE is numerically solved using a semi-implicit Euler method in time and centered finite differences in space to obtain the ground truth solutions. The resolution of solution space is fixed to the grid of 65×65. Similar to the previous example, we seek an operator that maps the initial condition $u_0(x)$ to a spatiotemporal solution at some time t, i.e., $D : u_0(x) \mapsto u(x,t)$. To generate the instances of varying initial conditions, we simulated $u_0(x)$ from a Gaussian Random Field (GRF) [48]. Thereafter, we used a finite difference-based script to obtain the ground truth. The results are illustrated in Fig. 3, whereas the comparative study is demonstrated in the Table 1.

The Fig. 3 envisions the predicted solution and the ground truth solution. The visual resemblance of the ground truth and predicted output and the prediction error plots over the domain corresponding to the 4 unseen realizations signify the excellent performance physics informed WNO. The results of physics-informed WNO, data-drive WNO, and hybrid data and physics informed are listed in Table 1. The observations from the table reinforce the capability of the physics WNO in learning functional mapping.

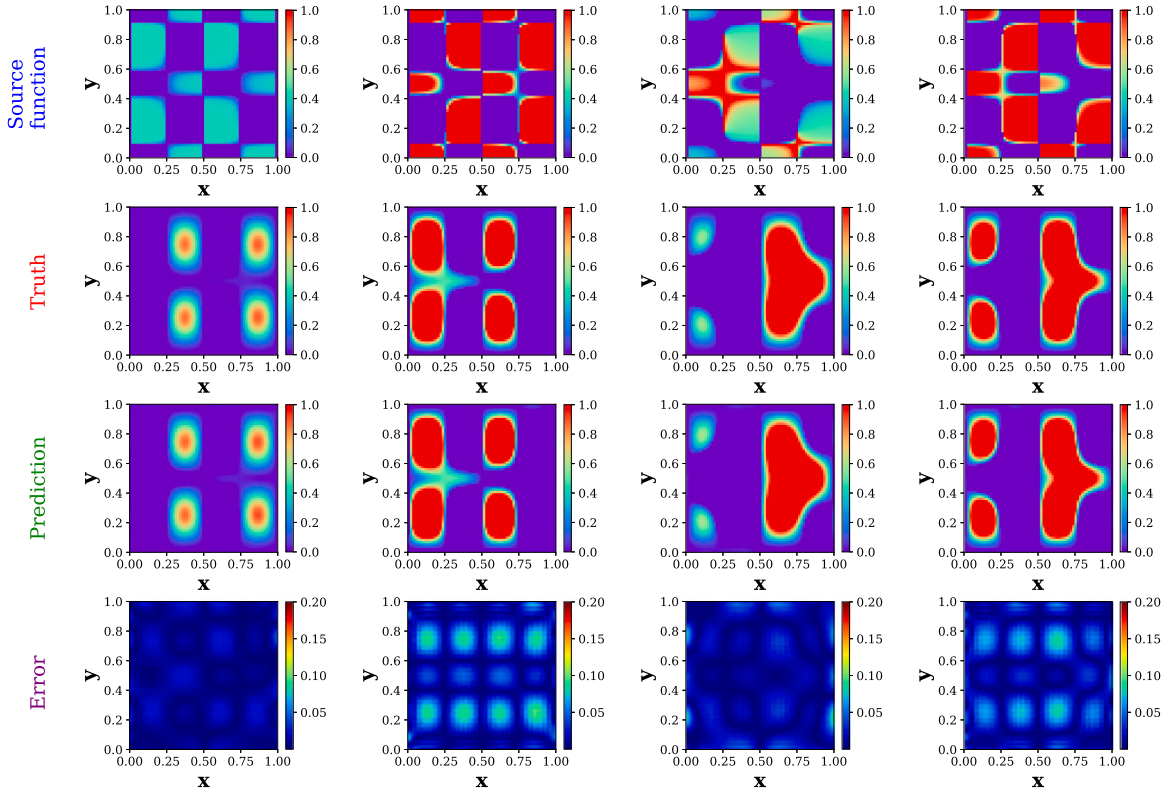

Fig. 4. The results for the Non-homogeneous Poisson's equation comprised of given source functions, corresponding ground truth solutions, predictions, and error plots illustrated with 4 representative sample instances out of 100 test samples. The physics-informed WNO maps the source term $f(x, y)$ to the corresponding solution $u(x, y)$ over the domain with a spatial resolution of 65×65.

However, contrary to the previous examples, the average relative MSE of the physics-informed WNO (0.0778%) is more than that of the data-drive WNO(0.0254%). WNO trained with both data loss and physics loss on the other hand results in better model generalization in comparison with the data-driven WNO with an average MSE of (0.0202%).

4.3. Non-homogeneous Poisson's equation

Non-homogeneous Poisson's equation is an elliptic PDE that has broad utilities in various physical situations and is considered to be the underlying mathematical model for problems, including steady-state heat diffusion and the eclectic field caused by a given electric charge. The Poisson's equation with a source term $f(x, y)$ and periodic boundary condition is expressed as:

$$\begin{aligned} &\partial_{xx}u + \partial_{yy}u = f(x, y), \quad x, y \in [-1, 1] \\ &u(x = -1, y) = u(x = 1, y) = u(x, y = -1) = u(x, y = 1) = 0 \end{aligned} \tag{31}$$

Here the problem under consideration is a static problem. In this example we aim to learn the operator, $\mathcal{D} : f(x, y) \mapsto u(x, y)$, where $u(x, y)$ is the solution of the equation over the 2-D domain. In other words here we attempt to learn operator mapping between the source function $f(x, y)$ to the solution $u(x, y)$. An analytical solution of the form $u(x, y) = \alpha \sin(\pi x)(1+\cos(\pi y))+\beta \sin(2\pi x)(1-\cos(2\pi y))$ is conveniently chosen to obtain the ground truth solution. As to generate input data, an analytical form of the source function is used such that $f(x, y) = 16\beta\pi^2 (\cos(4\pi y)\sin(4\pi x)+\sin(4\pi x)(\cos(4\pi y)-1))-\alpha\pi^2(\cos(\pi y)\sin(\pi x)+\sin(\pi x)(\cos(\pi y)+1))$. Here the expression of $f(x, y)$ is obtained by substituting the analytical expression of $u(x, y) = \alpha \sin(\pi x)(1 + \cos(\pi y)) + \beta \sin(2\pi x)(1 - \cos(2\pi y))$ in Eq. (31). Thus, the boundary conditions are implicitly satisfied the solution here. The samples of the training instances are yielded by varying the variable α and β such that $\alpha \sim \mathrm{Unif}(-2, 2), \eta \sim \mathrm{Unif}(-2, 2)$. The demonstrated results in Fig. 4 include the input function, ground truth solution, prediction, and prediction error. The results clearly indicate the efficacy of the proposed framework in learning the underlying PDE operator as predictions of the proposed framework in all 4 different test cases show an excellent agreement with the corresponding reference solutions. Furthermore, from the tabulated results shown in Table 1 physics-informed WNO yields an average relative MSE of 0.0469%, while the data-driven WNO yields a higher value of average relative MSE (0.548%). In this example as well, the hybrid training using data-driven and physics-informed achieves the highest accuracy.

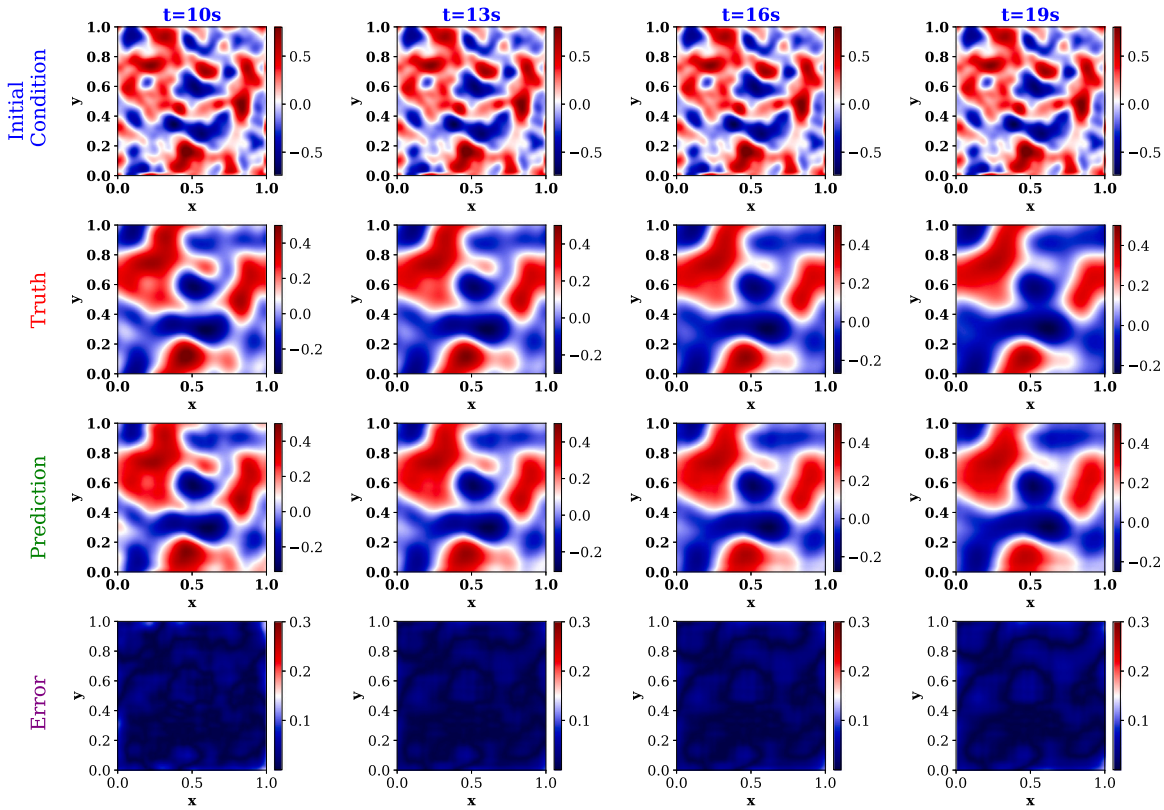

Fig. 5. The results for Allen–Cahn's equation comprised of a given initial field and corresponding ground truth solutions, predictions, and error plots illustrated with an unseen representative sample instance out of 60 test samples at time steps 10 s,13 s,16 s and 19 s. The physics-informed WNO receives the spatial field, $u(x, y)$ with a spatial resolution of 65×65, for the initial 10-time steps and which maps to the corresponding solution $u(x, y)$ for the 10 future time-steps.

4.4. Allen-Cahn equation

As the last example, we consider the Allen–Cahn equation, which is a well-studied partial differential equation describing the phenomenon of reaction–diffusion [47,49]. Applications of the mathematical model are not limited but can also be found in the context of phase separation in the multi-component alloy, chemical reactions, and crystal growth. The 2-D Allen–Cahn equation is expressed as:

$$\begin{aligned} \partial_t u(x,y,t) &= \epsilon \Delta u(x,y,t) + u(x,y,t) - u(x,y,t)^3, \quad && x,y \in [0,1] \\ u(x,y,0) &= u_0(x,y) && x,y \in [0,1] \end{aligned} \tag{32}$$

where $\epsilon \in \mathbb{R}^{+*}$ is a real positive viscosity coefficient. For the current study, ϵ is chosen to be $\epsilon = 1 \times 10^{-3}$. The problem is defined on a periodic boundary. The Gaussian Random Field with the following kernel is utilized to generate the initial conditions:

$$\mathcal{K}(x,y) = \tau^{(\alpha-1)} \left(\pi^2 \left(x^2 + y^2\right) + \tau^2\right)^{\frac{\pi}{2}}. \tag{33}$$

Here the parameters for the kernel are chosen as $\tau = 15$ and $\alpha = 1$. The spectral Galerkin method is used to generate the training and testing data [47]. In this example, we aim to learn the solution operator $\mathcal{D} : \boldsymbol{u}|_{[0,1]^2\times[0,10]} \mapsto \boldsymbol{u}|_{[0,1]^2\times[10,T]}$. More precisely, the operator $\mathcal{D}$ here maps the temporal fields, $u(x, y, t)$, at the time steps $t \in [0, 10]$ to the solution fields, $u(x, y, t)$, at a target time step $t \in [10, T]$. For the illustration, we predict the solution till the next 10 time steps, i.e., $T = 20$ steps. A visualization of the results is provided in Fig. 5, where the predicted solution and the ground truth at times steps $t = 10$ s, $t = 13$ s, $t = 16$ s and $t = 19$ s are compared. For the given unseen scenario, the prediction of the physics informed matches closely with the ground truth solution. Furthermore, it can be observed from the prediction error listed in the table that the average relative MSE of the physics-informed WNO is around 2.73%, which in comparison with previous examples, is a larger value. However, for this example, the data-driven WNO yields an average relative MSE of (2.98%), which is slightly a higher value than that of the physics-informed WNO. As seen in all previous examples, here as well, the WNO trained with both data loss and physics loss leads to greater model generalization with an average relative MSE of (2.37%).

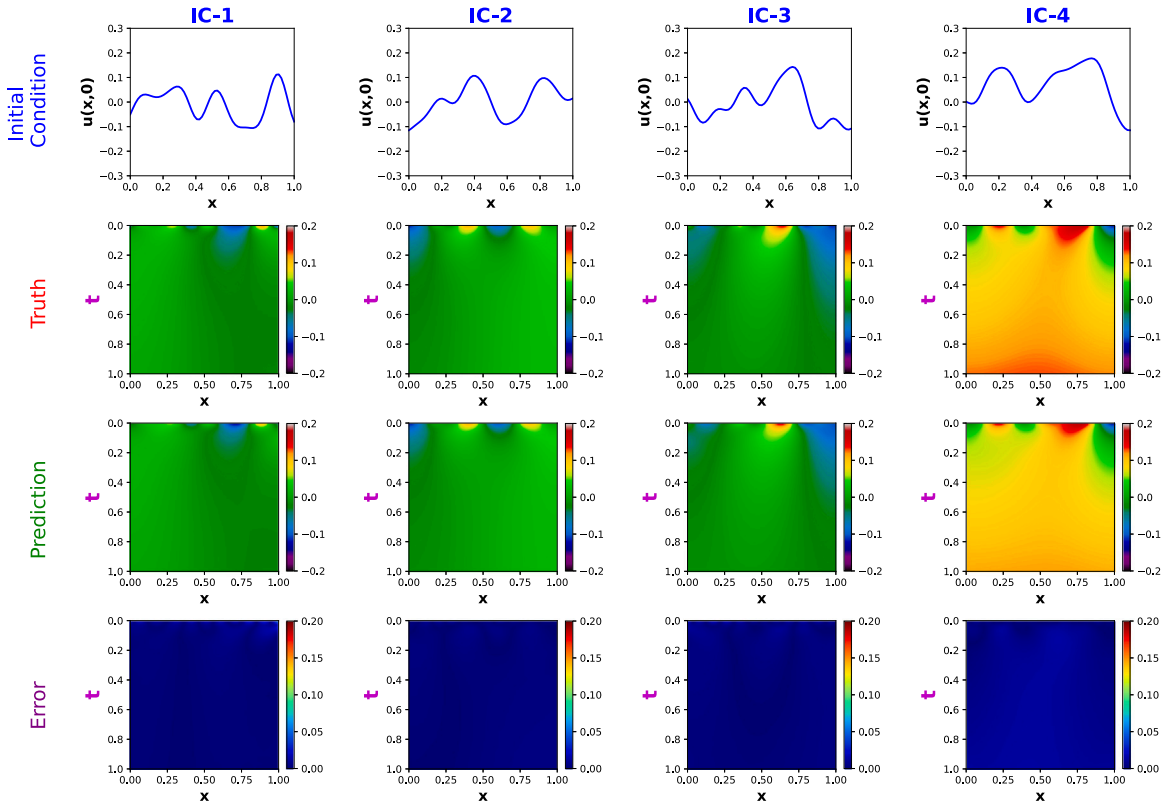

Fig. A.6. The results for the parametric Nagumo equation comprised of given initial conditions, corresponding ground truth solutions, predictions, and error plots illustrated with 4 different unseen representative sample instances(out of 100 test samples) for the case, where test data is generated from out of distribution, i.e kernal parameters are chosen such that ($\sigma = 0.08$, and $l = 0.08$). The physics-informed WNO maps the initial condition to the corresponding solution $u(x,t)$ over the domain with a Spatio-temporal resolution of 65×65.

5. Conclusions

In this paper, we propose physics-informed WNO (PIWNO), a novel physics-informed operator framework for approximating nonlinear operators which enables learning directly from the governing physics. Theoretically, to obtain the functional mapping in infinite dimensional Banach spaces, PIWNO performs a series of updates on input followed by local transformations. The operations are facilitated by the integral kernel operator in combination with nonlinear activation functions, while the parameterized integral kernel is devised as a global convolutional neural network. Further, the governing physics is conveniently incorporated by employing stochastic projection-based gradients. In essence, the proposed method is capable of learning the solutions even when the problem is nonlinear and complex, without any labeled data. PIWNO possesses unified wavelet integral layers that are able to track the finest features of signals through spatial and frequency localization. This allows the PIWNO to outperform the state of art operator frameworks in learning the highly nonlinear operators even when the solution domain has complex geometry and boundary conditions. While the vanilla WNO demands the labeled data or the PDE solution corresponding to the input functions, the PIWNO does not rely on the conventional PDE solvers for training. Moreover, the data-driven WNO can approximate the operator accurately only in the regimes of training data, whereas PIWNO can obtain a more generalized model. Once trained, a single PIWNO model can rapidly provide solutions to PDEs for different initial and boundary conditions and different input model parameters. On the contrary, conventional PDE solvers like finite element and finite difference and recently proposed physics-informed neural networks (PINN) require repeated independent runs for different input conditions.

It is noteworthy that the proposed approach is highly efficient and is ideally suited for PDEs governing real-time scenarios. The scope of the work is not limited but can be extended to interesting directions, such as applications of PIWNO for the development of digital twins and physics-augmented learning. Despite the promising performance of the PIWNO, it is liable to some significant drawbacks. Firstly, the absence of a standardized procedure to obtain the optimum and desired network architecture of a PIWNO and to assign the weights of boundary/initial condition loss. Secondly, the lack of information regarding how to learn the operator for systems involving multi-scale physics systems. To this end, we argue that the aforementioned challenges can be considered as the prospective goals, and addressing the limitations could result in a robust framework for the physical systems across a wide range of science and engineering applications.

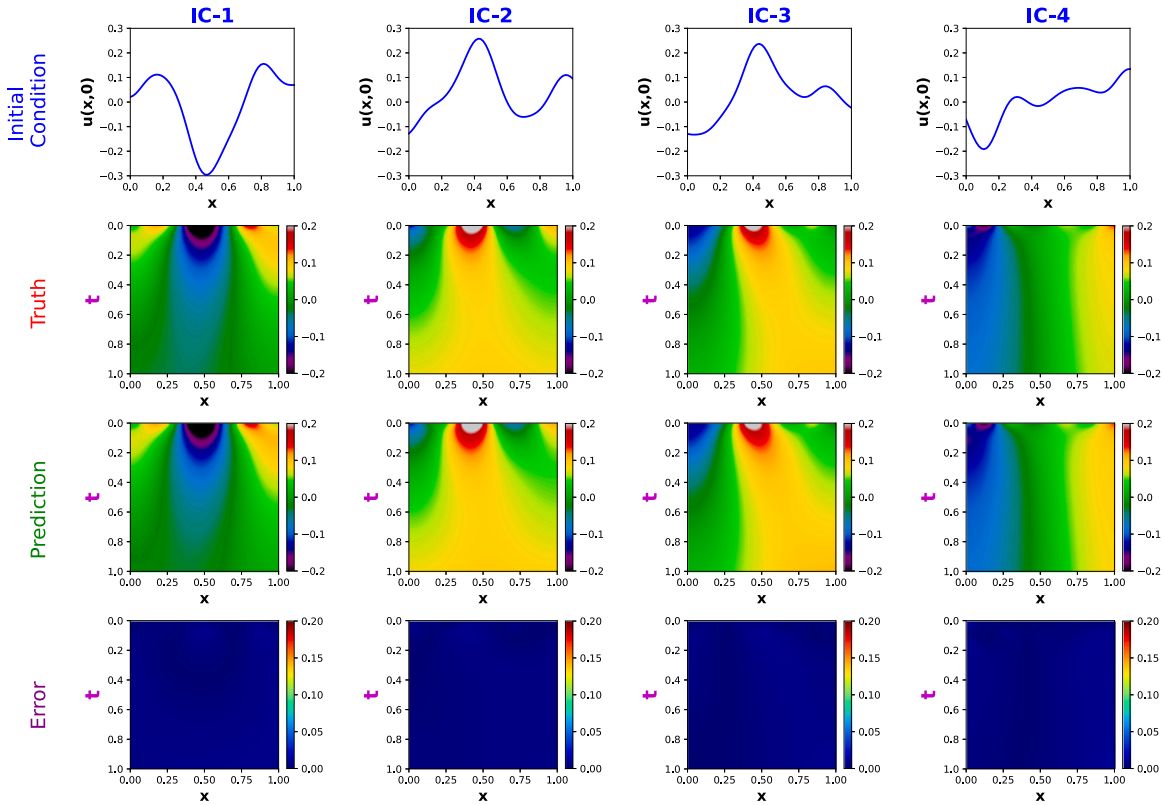

Fig. A.7. The results for the parametric Nagumo equation comprised of given initial conditions, corresponding ground truth solutions, predictions, and error plots illustrated with 4 different unseen representative sample instances(out of 100 test samples) for the case, where test data is generated from out of distribution, i.e kernal parameters are chosen such that ($\sigma = 0.12$, and $l = 0.12$). The physics-informed WNO maps the initial condition to the corresponding solution $u(x,t)$ over the domain with a Spatio-temporal resolution of 65×65.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Souvik Chakraborty reports financial support was provided by Science and Engineering Research Board.

Data availability

Data and codes associated with this paper can be accessed at https://github.com/csccm-iitd/Physics-informed-WNO.

Acknowledgments

NN and TT acknowledge the support received from the Ministry of Education (MoE), India, in the form of the Prime Ministers Research Fellowship (PMRF). SC acknowledges the financial support received from the Science and Engineering Research Board (SERB) via grant no. SRG/2021/000467, Ministry of Port and Shipping via letter no. ST-14011/74/MT (356529), and seed grant received from IIT Delhi, India.

Appendix. Prediction time comparison

A.1. Prediction time comparison

See Table A.2.

A.2. Results of out-of-distribution prediction

See Figs. A.6 and A.7.
See Tables A.3 and A.4.

Table A.2
Prediction time comparison of traditional solvers with the trained physics informed WNO for Burger's equation and Allen–Cahn Equation examples.

	Burger's equation	Allen–Cahn
	Prediction time	Prediction time
Traditional solvers	337.54	2638.38
Trained WNO	**23.76**	**72.76**

Table A.3
Prediction results of the physics informed WNO for burgers examples for out-of-distribution cases, where the testing data is generated such that the parameters, ζ and η (Section 4.1) are simulated from distributions (a) $\zeta \sim \text{Trunc. Normal}(0.5, 1.5)$ and $\eta \sim \text{Trunc. Normal}(0.5, 1.5)$ (b) $\zeta \sim \text{Trunc. Normal}(0.4, 1.6)$ and $\eta \sim \text{Trunc. Normal}(0.4, 1.6)$.

	Distributions(ζ, η)	Range	Relative MSE
(a) Out of distribution	Truncated Normal	(0.5,1.5)	0.5556 ± 1.0310%
(b) Out of distribution	Truncated Normal	(0.4,0.16)	0.9058 ± 1.6470%

Table A.4
Prediction results of the physics informed WNO for burgers examples for out-of-distribution cases, where the testing data is generated such that the kernel parameters of the Gaussian Random Field (GRF), σ and l (Section 4.2) are given as (a) $\sigma = 0.08$, and $l = 0.08$ (b) $\sigma = 0.12$, and $l = 0.12$.

	Kernal parameters		Relative MSE
	σ	l	
(a) Out of distribution	0.08	0.08	1.3528 ± 0.4139%
(b) Out of distribution	0.12	0.12	0.6864 ± 0.2920%

References

[1] L. Debnath, L. Debnath, Nonlinear Partial Differential Equations for Scientists and Engineers, Springer, 2005.
[2] D.S. Jones, M. Plank, B.D. Sleeman, Differential Equations and Mathematical Biology, Chapman and Hall/CRC, 2009.
[3] L.C. Evans, Partial Differential Equations. Vol. 19, American Mathematical Soc, 2010.
[4] F. Kang, S. Zhong-Ci, F. Kang, S. Zhong-Ci, Finite element methods, Mathematical Theory of Elastic Structures (1996) 289–385.
[5] J.A. Cottrell, T.J. Hughes, Y. Bazilevs, Isogeometric Analysis: Toward Integration of CAD and FEA, John Wiley & Sons, 2009.
[6] M.N. Özişik, H.R. Orlande, M.J. Colaco, R.M. Cotta, Finite Difference Methods in Heat Transfer, CRC Press, 2017.
[7] R. Eymard, T. Gallouët, R. Herbin, Finite volume methods, Handb. Num. Anal. 7 (2000) 713–1018.
[8] J. Sirignano, K. Spiliopoulos, DGM: A deep learning algorithm for solving partial differential equations, J. Comput. Phys. 375 (2018) 1339–1364.
[9] Q. Chan-Wai-Nam, J. Mikael, X. Warin, Machine learning for semi linear PDEs, J. Sci. Comput. 79 (3) (2019) 1667–1712.
[10] E. Samaniego, C. Anitescu, S. Goswami, V.M. Nguyen-Thanh, H. Guo, K. Hamdia, X. Zhuang, T. Rabczuk, An energy approach to the solution of partial differential equations in computational mechanics via machine learning: Concepts, implementation and applications, Comput. Methods Appl. Mech. Engrg. 362 (2020) 112790.
[11] D.C. Psichogios, L.H. Ungar, A hybrid neural network-first principles approach to process modeling, AIChE J. 38 (10) (1992) 1499–1511.
[12] I.E. Lagaris, A. Likas, D.I. Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, IEEE Trans. Neural Netw. 9 (5) (1998) 987–1000.
[13] L. Sun, H. Gao, S. Pan, J.-X. Wang, Surrogate modeling for fluid flows based on physics-constrained deep learning without simulation data, Comput. Methods Appl. Mech. Engrg. 361 (2020) 112732.
[14] Y. Zhu, N. Zabaras, P.-S. Koutsourelakis, P. Perdikaris, Physics-constrained deep learning for high-dimensional surrogate modeling and uncertainty quantification without labeled data, J. Comput. Phys. 394 (2019) 56–81.
[15] S. Karumuri, R. Tripathy, I. Bilionis, J. Panchal, Simulator-free solution of high-dimensional stochastic elliptic partial differential equations using deep neural networks, J. Comput. Phys. 404 (2020) 109120.
[16] K. Wu, D. Xiu, Data-driven deep learning of partial differential equations in modal space, J. Comput. Phys. 408 (2020) 109307.
[17] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, J. Comput. Phys. 378 (2019) 686–707.
[18] S. Cai, Z. Mao, Z. Wang, M. Yin, G.E. Karniadakis, Physics-informed neural networks (PINNs) for fluid mechanics: A review, Acta Mech. Sin. 37 (12) (2021) 1727–1738.
[19] S. Chakraborty, Transfer learning based multi-fidelity physics informed deep neural network, J. Comput. Phys. 426 (2021) 109942.
[20] K. Hornik, M. Stinchcombe, H. White, Multilayer feedforward networks are universal approximators, Neural Netw. 2 (5) (1989) 359–366.
[21] T. Chen, H. Chen, Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems, IEEE Trans. Neural Netw. 6 (4) (1995) 911–917.
[22] C.C. Margossian, A review of automatic differentiation and its efficient implementation, Wiley Interdisc. Rev.: Data Min. Knowl. Discov. 9 (4) (2019) e1305.
[23] S. Goswami, C. Anitescu, S. Chakraborty, T. Rabczuk, Transfer learning enhanced physics informed neural network for phase-field modeling of fracture, Theor. Appl. Fract. Mech. 106 (2020) 102447.
[24] E. Kharazmi, Z. Zhang, G.E. Karniadakis, Hp-VPINNs: Variational physics-informed neural networks with domain decomposition, Comput. Methods Appl. Mech. Engrg. 374 (2021) 113547.
[25] L. Yuan, Y.-Q. Ni, X.-Y. Deng, S. Hao, A-PINN: Auxiliary physics informed neural networks for forward and inverse problems of nonlinear integro-differential equations, J. Comput. Phys. 462 (2022) 111260.
[26] X. Li, C. Gong, L. Gu, W. Gao, Z. Jing, H. Su, A sequential surrogate method for reliability analysis based on radial basis function, Struct. Saf. 73 (2018) 42–53.

[27] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, A. Anandkumar, Neural operator: Learning maps between function spaces, 2021, arXiv preprint arXiv:2108.08481.
[28] L. Lu, P. Jin, G.E. Karniadakis, Deeponet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators, 2019, arXiv preprint arXiv:1910.03193.
[29] L. Lu, P. Jin, G. Pang, Z. Zhang, G.E. Karniadakis, Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators, Nat. Mach. Intell. 3 (3) (2021) 218–229.
[30] S. Garg, S. Chakraborty, VB-DeepONet: A Bayesian operator learning framework for uncertainty quantification, Eng. Appl. Artif. Intell. 118 (2023) 105685.
[31] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, A. Anandkumar, Neural operator: Graph kernel network for partial differential equations, 2020, arXiv preprint arXiv:2003.03485.
[32] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, A. Anandkumar, Fourier neural operator for parametric partial differential equations, 2020, arXiv preprint arXiv:2010.08895.
[33] G. Wen, Z. Li, K. Azizzadenesheli, A. Anandkumar, S.M. Benson, U-FNO—An enhanced Fourier neural operator-based deep-learning model for multiphase flow, Adv. Water Resour. 163 (2022) 104180.
[34] T. Tripura, S. Chakraborty, Wavelet neural operator for solving parametric partial differential equations in computational mechanics problems, Comput. Methods Appl. Mech. Engrg. 404 (2023) 115783.
[35] A. Thakur, T. Tripura, S. Chakraborty, Multi-fidelity wavelet neural operator with application to uncertainty quantification, 2022, arXiv preprint arXiv:2208.05606.
[36] T. Tripura, A. Awasthi, S. Roy, S. Chakraborty, A wavelet neural operator based elastography for localization and quantification of tumors, Comput. Methods Programs Biomed. 232 (2023) 107436.
[37] D. Zhang, X. Han, C. Jiang, J. Liu, Q. Li, Time-dependent reliability analysis through response surface method, J. Mech. Des. 139 (4) (2017) 041404.
[38] A. Boggess, F.J. Narcowich, A First Course in Wavelets with Fourier Analysis, John Wiley & Sons, 2015.
[39] N. Navaneeth, S. Chakraborty, Stochastic projection based approach for gradient free physics informed learning, Comput. Methods Appl. Mech. Engrg. 406 (2023) 115842.
[40] V. Hutson, J. Pym, M. Cloud, Applications of Functional Analysis and Operator Theory, Elsevier, 2005.
[41] I. Daubechies, Ten Lectures on Wavelets, SIAM, 1992.
[42] F. Cotter, Uses of complex wavelets in deep convolutional neural networks (Ph.D. thesis), University of Cambridge, 2020.
[43] I.W. Selesnick, R.G. Baraniuk, N.C. Kingsbury, The dual-tree complex wavelet transform, IEEE Signal Process. Mag. 22 (6) (2005) 123–151.
[44] S. Kutluay, A. Bahadir, A. Özdeş, Numerical solution of one-dimensional Burgers equation: Explicit and exact-explicit finite difference methods, J. Comput. Appl. Math. 103 (2) (1999) 251–261.
[45] A.-M. Wazwaz, Partial Differential Equations, CRC Press, 2002.
[46] C. Laing, G.J. Lord, Stochastic Methods in Neuroscience, OUP Oxford, 2009.
[47] G.J. Lord, C.E. Powell, T. Shardlow, An Introduction to Computational Stochastic PDEs, Vol. 50, Cambridge University Press, 2014.
[48] C.M. Bishop, N.M. Nasrabadi, Pattern Recognition and Machine Learning, Vol. 4, Springer, 2006.
[49] L. Ma, R. Chen, X. Yang, H. Zhang, Numerical approximations for Allen-Cahn type phase field model of two-phase incompressible fluids with moving contact lines, Commun. Comput. Phys. 21 (3) (2017) 867–889.